Diagram Editing with Mermaid

Nastase Maria Magdalena Tudor Florina Iuliana

May 2026

1 Introduction

Diagrams are a common way to document software systems. They show how components are connected, how users interact with an application, or how data flows between services. In professional projects, such drawings often have to be updated many times. When diagrams are stored only as binary images or in proprietary formats, it is difficult to see what changed between two versions and to review updates in the same way as code changes.

Diagram *editing* means the whole process of creating, changing, checking, and publishing a diagram. With Mermaid, editing is done on a text file. The author writes or modifies lines that describe nodes, links, and messages. A renderer then produces the visual diagram, usually in SVG or PNG format (mermaid-js 2024). This approach is sometimes called *diagrams as code* (Lieberman 2021, Rule of Tech 2018). The important point for our case study is that the editable artifact is text, not a collection of shapes drawn with the mouse.

We chose the topic *Diagram Editing with Mermaid* because we wanted to understand not only what Mermaid can draw, but *how* a student or developer actually edits diagrams day to day: which editor to use, how preview works, what happens when syntax is wrong, and how the result can be placed in a report. We used Visual Studio Code with the Mermaid Preview extension and the Mermaid Live Editor.

Mermaid is implemented in JavaScript and is widely used in Markdown documentation on GitHub and GitLab (GitHub Docs 2024). It does not require installing Java, which distinguishes it from PlantUML, another text based tool discussed in the course. Mermaid is not a full model driven engineering environment: it does not define a metamodel, constraints, or transformations to programming languages in the sense of EMF or GME. Instead, it supports *notation editing* and communication. The objective of this report is to describe diagram editing with Mermaid in a structured way. Section 2 explains the editing model and the operations we performed on text. Section 3 compares Mermaid with PlantUML and with graphical tools. Section 4 presents our login flow example, including two diagram types and an error situation. Section 5 concludes.

2 Editing Diagrams with Mermaid

2.1 Source file and diagram declaration

Editing always starts with a plain text file. The first meaningful line declares the diagram type. For example, `flowchart TD` starts a top down flowchart, and `sequenceDiagram` starts a sequence diagram (Mermaid 2024). If this keyword is misspelled, the tool cannot recognise the diagram and editing stops at a parse error until the line is corrected. We illustrate such a situation in Figure 3 in Section 4.

2.2 Basic editing operations

Once the type is declared, editing consists of several recurring operations that we applied in our login example.

Adding and renaming nodes. Nodes are identified in the text and may have a label shown in the figure. In a flowchart, a rectangle is written as `B[Login Page]`, a decision as `C{Valid credentials?}`, and a rounded start shape as `A([User])`. To rename a step, only the label in brackets is changed; the internal id (`A`, `B`) can stay the same so that links do not break.

Adding links. Directed edges use arrows such as `->`. Labels on edges are written as `|yes|` or `|no|` between the nodes. Editing a flow often means adding or removing one line per link.

Grouping with subgraphs. Related nodes can be placed inside `subgraph Auth["Auth Service"] ... end`. This was important in our flowchart to show that validation belongs to the authentication component.

Styling. Commands like `classDef` assign colours or borders to nodes. We used styling in early versions of our diagram; the final figure in this report uses a simplified layout exported to PNG.

Comments. Lines starting with `%%` are ignored by the renderer but help the author and reviewers understand the file during editing.

2.3 Preview, correction, and export

A typical editing session follows a cycle. The author changes the text, saves the file, and looks at the preview. In Visual Studio Code, extensions such as Mermaid Preview show the diagram beside the source. On the web, the Mermaid Live Editor offers the same behaviour without a local project. When syntax is valid, the picture updates in a few seconds. When it is not, an error message points to the problem, similar to a compiler error for code.

For inclusion in a PDF report or a wiki, the diagram can be exported to PNG or SVG. We used the Mermaid CLI to generate the PNG files for the LaTeX report.

2.4 Diagram types we used while editing

The official documentation lists many diagram types (Mermaid 2024, Shifflett 2023). In this case study we edited two of them for the same login scenario: a **flowchart** for the control flow from the

user perspective, and a **sequence diagram** for messages between user, web application, authentication service, and database. Flowcharts are well suited to decisions and branches; sequence diagrams show the order of calls and return values. Editing the second diagram required learning different keywords (`participant`, `-»`, `alt`, `else`), which took more time than editing the flowchart, but the preview cycle was the same.

Other types mentioned in the literature, such as class diagrams, state diagrams, or Gantt charts, follow the same text based editing idea. Features for C4 architecture diagrams are still evolving; some authors prefer PlantUML for that purpose (Does Code 2024).

2.5 Where we edited the diagrams

We used three environments. First, Visual Studio Code with live preview for most of the writing. Second, the Mermaid Live Editor for quick tests. Third, Markdown preview for `login-flow-demo.md`, which shows how the same diagram appears when embedded in documentation on GitHub (GitHub Docs 2024). For the LaTeX report, only the exported PNG files are embedded; the source remains in the `example/` folder for reproducibility.

3 Comparative Analysis

3.1 Comparison with PlantUML

PlantUML is another text based diagram tool (Diagrams.so 2024, Revision 2025, Seekatar 2024). It offers a large set of UML diagram types and detailed control over sequence diagrams, for example numbered messages and nested interaction frames. Mermaid is easier to start with for students because it runs in the browser and integrates directly into GitHub Markdown. Table 1 summarises aspects related to editing over time. Both tools keep diagrams as text in Git; neither replaces Papyrus or GME for metamodel based development.

Table 1: Comparison of editing aspects for Mermaid and PlantUML (based on Revision 2025, Wong 2025).

Aspect	**Mermaid**	**PlantUML**
Installation	Browser or npm; no Java	Java and often a server
Use in GitHub Markdown	Supported directly	Usually needs a plugin
UML notation	Practical, less formal	Closer to UML standard
Number of diagram types	Increasing	Larger set
Layout options	dagre, ELK optional	More manual control
Storage in Git	Text files	Text files

3.2 Comparison with graphical and AI based tools

Tools such as Visual Paradigm, Microsoft Visio, or Eraser focus on drawing with the mouse. They are convenient for slides, but comparing versions in Git is harder. Mermaid requires learning syntax, yet each edit appears as a line change in a diff. AI tools can suggest diagram content, but we think the result still needs manual cleanup and storage as Mermaid or PlantUML text if the team wants to maintain it in the repository.

4 Practical Work: Editing a Login Flow

4.1 Description of the example

We modelled a login process for a web application. The flowchart in `example/login-flow.mmd` shows the user, the login page, validation inside an `Auth Service` subgraph, and the outcomes: create session or show error. The sequence diagram in `example/login-flow-sequence.mmd` describes the same story as message exchanges with the database and alternative blocks for success and failure.

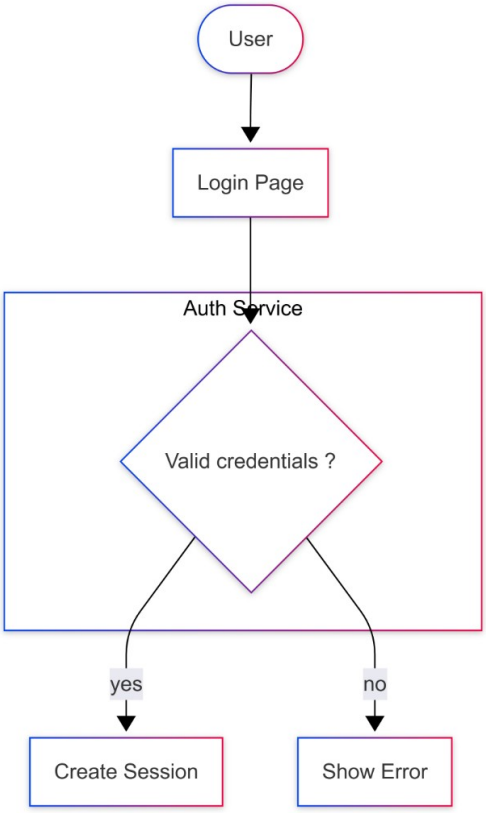

Figure 1: Flowchart of the login flow (`login-flow.mmd`).

Figures 1 and 2 show the final results. Editing the flowchart took fewer lines of text; editing the sequence diagram required defining participants and an `alt` block for valid and invalid credentials.

Listing 1 shows the flowchart source. Listing 2 shows an excerpt of the sequence diagram.

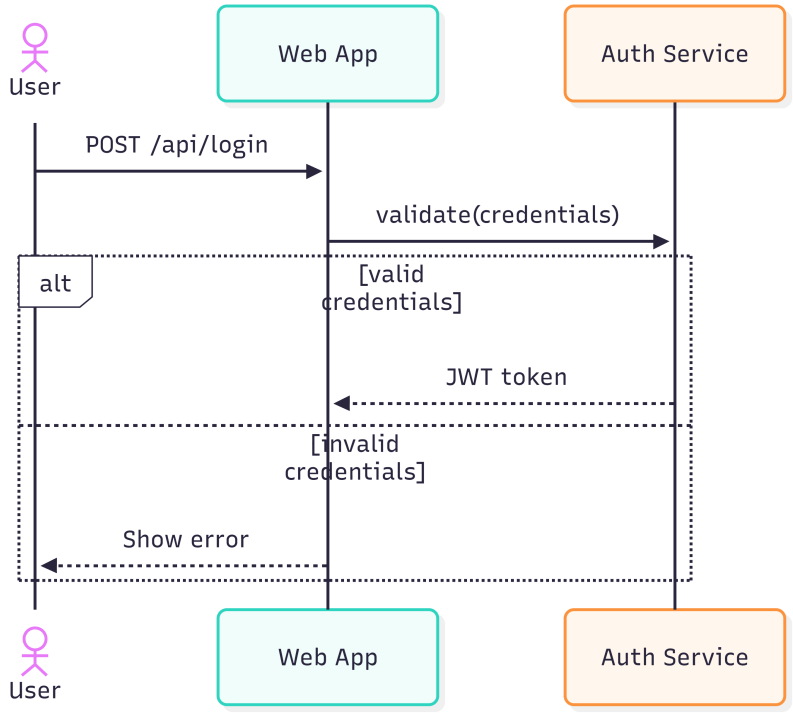

Figure 2: Sequence diagram of the login flow (`login-flow-sequence.mmd`).

Listing 1: Flowchart source (`login-flow.mmd`)

```
flowchart TD
    A([User]) --> B[Login Page]
    B --> C{Valid credentials?}
    C -->|yes| D[Create Session]
    C -->|no| E[Show Error]
    subgraph Auth["Auth Service"]
        C
    end
```

Listing 2: Sequence diagram excerpt (`login-flow-sequence.mmd`)

```
sequenceDiagram
    actor U as User
    participant W as Web App
    participant A as Auth Service
    U->>W: POST /api/login
    W->>A: validate(credentials)
    alt valid credentials
        A-->>W: JWT token
    else invalid credentials
        W-->>U: Show error
    end
```

4.2 Steps we followed while editing

We started with a longer flowchart that included extra steps such as opening the page and submitting a form as separate boxes. After previewing the layout, we removed steps so the diagram fits on one page and remains readable. We then added the `Auth Service` subgraph around the decision node. For the sequence diagram, we wrote the happy path first and added the `else` branch for invalid credentials second. After each change we saved the file and checked the preview.

We exported PNG images for this report. The flowchart PNG was prepared with our preferred visual style; the sequence diagram PNG was generated with `mmdc`. Both sources remain in the `example/` directory so that anyone can reproduce or continue editing.

4.3 Syntax error during editing

An important part of editing is handling mistakes. We deliberately wrote `flowchat` instead of `flowchart` in file `login-flow-error.mmd`. The preview did not show a diagram; instead, the tool reported a parse error. Figure 3 reproduces the situation: the wrong keyword is visible in the source, and the error panel explains that the diagram type is unknown.

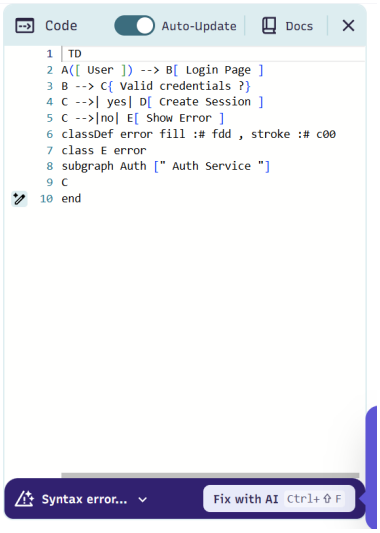

Figure 3: Parse error when the diagram keyword is misspelled (`flowchat` instead of `flowchart`).

This experience shows that Mermaid editing depends on correct syntax. The feedback is fast, which encourages a short edit-preview cycle. In a team project, the same file could be validated in continuous integration by running `mmdc` and failing the build if the diagram does not render.

4.4 Difficulties observed

When many nodes were present, automatic layout sometimes drew crossing edges. Documentation describes optional ELK layout for larger graphs, but it requires extra configuration. For sequence diagrams, we missed automatic message numbering, which PlantUML provides (Wong 2025). Special characters in labels, such as slashes in a URL path, must be quoted or escaped. These issues did not block our example, but they limit use for very large or highly formal specifications.

5 Conclusion

This report analysed diagram editing with Mermaid for the MDSE case study. We combined documentary research with practical editing of a login flow example. The following conclusions summarise what we learned from the editing work itself and from the comparison with other tools.

5.1 What we learned from editing

During the practical part, we understood that editing a Mermaid diagram is essentially editing a small program that describes structure. The first line matters: without a valid diagram type, nothing is rendered, as we saw when we wrote `flowchat` instead of `flowchart` (Figure 3). This was useful because it showed that mistakes are detected early, before the diagram is published.

We also learned that a stable editing routine works well. We changed the text, saved the file, checked the preview, and repeated until the layout was clear. Keeping short node identifiers (`A`, `B`, `C`) while changing labels made refactoring easier. Subgraphs helped organise the flowchart without adding more arrows. For the sequence diagram, the syntax was longer, but the same preview cycle applied; writing the main path first and the `alt/else` branch second reduced confusion.

Finally, we noticed limits of automatic layout. A simple diagram with few nodes was easy to control; a larger diagram would need more time or optional layout engines. Special characters in labels require care. These points do not prevent use for documentation, but they show that Mermaid editing is not the same as freehand drawing with unlimited placement.

5.2 What we learned from the comparison

The comparison in Section 3 and in Table 1 led us to separate tools by purpose rather than to rank one as always better.

Mermaid. We would choose Mermaid when diagrams must appear in Markdown on GitHub, when the team wants a quick start without Java, and when flowcharts or sequence diagrams are enough. The editing experience felt accessible for students: little installation, immediate preview, and readable source files.

PlantUML. Literature and our own reading (Revision 2025, Wong 2025, Does Code 2024) suggest PlantUML when stricter UML notation is required, when sequence diagrams need richer features such as numbered messages, or when more diagram types are needed in one toolchain. The cost is a heavier setup (Java, server, or CLI in continuous integration).

References

Diagrams.so (2024) *Diagram as Code Comparison: Mermaid, PlantUML, D2.* Available at: `https://diagrams.so/learn/diagram-as-code-comparison` (Accessed: 25 May 2026).

Does Code, D. (2024) *Software Diagrams: PlantUML vs Mermaid.* Available at: `https://www.dandoescode.com/blog/plantuml-vs-mermaid` (Accessed: 25 May 2026).

GitHub Docs (2024) *Creating diagrams.* Available at: https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams (Accessed: 25 May 2026).

Lieberman, T. (2021) *Diagrams as Code: PlantUML or Mermaid.* Available at: https://wyssmann.com/blog/2021/03/diagrams-as-code-plantuml-or-mermaid/ (Accessed: 25 May 2026).

Mermaid (2024) *Diagram Syntax Reference.* Available at: https://mermaid.js.org/intro/syntax-reference.html (Accessed: 25 May 2026).

mermaid-js (2024) *mermaid* GitHub repository. Available at: https://github.com/mermaid-js/mermaid (Accessed: 25 May 2026).

Revision (2025) *Mermaid vs PlantUML: Which Tool Fits Best?* Available at: https://revision.app/blog/mermaid-vs-plantuml (Accessed: 25 May 2026).

Rule of Tech (2018) *Generating documentation as code with mermaid and PlantUML.* Available at: https://ruleoftech.com/2018/generating-documentation-as-code-with-mermaid-and-plantuml (Accessed: 25 May 2026).

Seekatar (2024) *plantuml-mermaid* comparison repository. Available at: https://github.com/Seekatar/plantuml-mermaid (Accessed: 25 May 2026).

Shifflett, B. (2023) Mermaid Chart: Architecture Diagrams. Available at: https://mermaid.js.org/ (Accessed: 25 May 2026).

Wong, J. (2025) *PlantUML vs Mermaid: Sequence Diagrams in Markdown.* Available at: https://jimmywongiot.com/2025/08/28/plantuml-vs-mermaid-which-tool-is-best-for-sequence-diagrams-in-markdown/ (Accessed: 25 May 2026).